

EST-1141

Lenguajes de Programación

Juan Zamora O.

Agosto, 2024.



Estructura

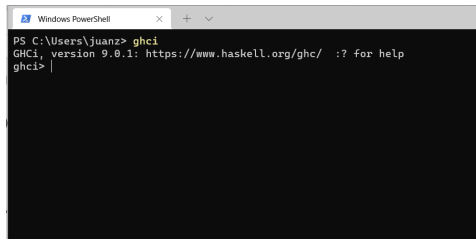
- 1 Introducción a Haskell
- 2 Acerca de la importancia de los Tipos
- 3 Tipos de dato definidos por el Usuario
- 4 Entrada y Salida en Haskell

¿Que es Haskell?

- Lenguaje funcional puro
- Tareas expresadas en términos de funciones
- Una función solo calcula usando los parámetros y retorna un determinado valor
 - No existe efecto lateral (side effect)
- Es perezoso: Postpone la evaluación de una expresión hasta que se necesita el resultado para otros cálculos
 - Opuesto: evaluación impaciente (eager)
- Evaluación perezoso (lazy)
- Permite pensar programas como series de transformaciones sobre datos
- Es estáticamente tipado
 - Tipos de expresiones son resueltos al compilar un programa
 - Varios errores son oportunamente detectados
- Tiene inferencia de tipos

El entorno de Haskell

- Haskell tiene varias implementaciones (Hugs, GHC...)
- Usaremos el ***Glasgow Haskell Compile*** (GHC)
- GHC tiene 3 componentes:
 - ghc: Compilador que genera código nativo
 - ghci: Interprete interactivo
 - runghc: Programa para ejecutar scripts en Haskell sin tener que compilarlos



```
Windows PowerShell
PS C:\Users\juanz> ghci
GHCi, version 9.0.1: https://www.haskell.org/ghc/  :? for help
ghci> |
```

Interacciones básicas con el Interprete

-- *aritmética simple, notación infija y prefija*

2 + 7

(+) 2 7 -- *probar sin paréntesis*

2 * 3

2 * (-3) -- *probar sin paréntesis*

(*) 4 8 -- ** es una función?*

succ 5

min 9 10

max 7 9 -- *cómo calcular el max entre mas de 2 números. Probar que pasa con*

(succ 9) + (max 5 4) + 1

-- *Lógica booleana, operadores y comparación de valores*

True && False

False || True -- *probar reemplazando True con 1*

-- *Operadores de comparación*

1 == 1

2 < 3

4 >= 3.99

5 /= 6 -- *En Haskell no existe !=*

not True

Precedencia y Asociatividad

1 + 4 * 4

5 - 10 / 2

5 - (div 10 2)

5 - div 10 2

5 + 10 + 2 -- de izq a der es decir (5 + 10) + 2

Precedence	Operator	Description	Associativity
9 highest	.	Function composition	Right
8	^, ^^, **	Power	Right
7	*, /, 'quot', 'rem', 'div', 'mod'		Left
6	+, -		Left
5	:	Append to list	Right
4	==, /, <, <=, >=, >	Compare-operators	
3	&&	Logical AND	Right
2		Logical OR	Right
1	>>, >>=		Left
1	<<=		Right
0	\$/, \$!, 'seq'		Right

EST-1141

Listas

- Colecciones homogéneas

[1, 2, 3]

["aba", "bba", "aab"]

[1..5]

[11, 9..1]

[10, 9..1]

- Operadores sobre listas ++ y :

```
[3,1,3] ++ [4,5]
```

```
[] ++ [False, True] ++ [True]
```

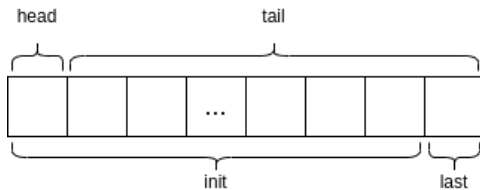
```
1 : []
```

```
1 : [2,3] -- Intente cambiar los operandos de orden
```

A continuación se presentan las operaciones más usadas sobre listas, junto con un esquema que muestra las 4 fundamentales:

head
tail
last
init
length
null
reverse
take
drop
maximum
sum
product
elem
zip

Queda como responsabilidad de l@s estudiantes averiguar cómo se usa y qué hace cada una



Cadenas y caracteres

- Cadenas se definen mediante comillas dobles
- Caracteres se definen mediante comillas simples

```
"hola"
```

```
println "Este es un mensaje"
```

```
println ['h','o','l','a'] -- Una cadena es una lista de caracteres
```

```
'h' : "ola"
```

```
'h' : ['o','l','a']
```

Compilando nuestro primer archivo

- 1 Cree un archivo `hola_curso.hs` en la carpeta que usted elija
- 2 Agregue el contenido:

```
main = do
  putStrLn "Hola curso" -- ojo con la tabulación.
  putStrLn "Adios."
```

- Ejecute desde la consola (dentro del mismo directorio anterior)

```
ghc hola_curso.hs
```

```
./hola_curso
```

Actividades

- Ingrese las siguientes expresiones en `ghci` e indique el tipo asociado (`:type XX`):

`5 + 8`

`3 * 5 + 8`

`2 + 4`

`(+) 2 4`

`sqrt 16`

`succ 6`

`pred 9`

`sin(pi / 2)`

`truncate pi`

`round 3.7`

`ceiling 3.5`

- Cree el archivo `ciudades.txt` con el siguiente contenido:

Santiago, Chile

Buenos Aires, Argentina

Montevideo, Uruguay

Brasilia, Brasil

Quito, Ecuador

- contador.hs y otro archivo denominado

-- *archivo contador.hs*

```
main = interact wordCount
  where wordCount input = show (length (lines input)) ++ "\n"
```

- Compile el archivo hs mediante el comando `ghc contador.hs`
- Si resulta todo **OK**, ejecute `./contador < ciudades.txt` ¿Qué está contando el programa?

Acerca de la importancia de los Tipos

- Toda expresión en Haskell tiene un tipo
- Por una parte este tipo **indica** que el valor asociado **comparte** ciertas **propiedades** con otros valores del mismo tipo
- Por ejemplo, podemos sumar números o concatenar listas
- Diremos siempre que una expresión tiene un tipo X p es del tipo X

¿Para qué sirven los tipos en un LP?

- A nivel de hardware un computador procesa bytes
- Un sistema de tipos nos:
 - Entrega **Abstracción**
 - Si sé que un valor de mi programa es de tipo String, ***no me interesa o afecta*** los detalles de cómo se representa, almacena o recupera ese valor desde el hardware
 - Agrega significado a esos bytes. ***Estos bytes corresponden a texto, una reserva de la aerolínea, información personal de un paciente ...***
 - Ayuda a prevenir la mezcla incoherente de tipos
- Un sistema de tipos permite delinear la manera en que pensamos y escribimos código en el Lenguaje
- El sistema de tipos de Haskell permite pensar en un nivel muy abstracto, permite además ***escribir*** programas concisos y efectivos

El sistema de tipos de Haskell

- Fuerte
- Estático
- Con inferencia automática

Tipado Fuerte

- Sistema garantiza que un programa no puede contener **ciertos tipos de error**
- ¿Que errores?
 - Formar expresiones sin sentido
 - E.g. Usar una función como un número entero o entregarle un String a una función que espera un valor flotante.
 - Convertir un valor de un tipo a otro. Haskell no convierte automáticamente tipos de valores. (Esto no siempre es lo más conveniente en términos de eficiencia)
- Un sistema con tipado fuerte permite atrapar errores difíciles de detectar en el código antes de que causen problemas
- Un sistema con tipado fuerte tratará como válidas una **menor cantidad** de expresiones en comparación con otro sistema más frágil (**weak**)

Tipado Estático

- Esto significa que el compilador conoce el tipo de cada valor y expresión al compilar el código (Antes generar el ejecutable)
- Esto genera programas que difícilmente se caeran por errores triviales
- Por ejemplo, al intentar usar la expresión `True && "False"`, el compilador infiere los tipos y detecta que **no** es posible emparejarlos

`True && "False"`

- Otros lenguajes (como Python) usan el denominado ***Duck Typing***
 - El tipo asociado a un objeto no es tan importante como la compatibilidad con la operación que se está intentando realizar
 - Los tipos no se verifican sino que se verifica la existencia de métodos o atributos que permitan la evaluación de una expresión

```
In [6]: def calcular(a,b,c):  
...:     return (a + b) * c  
...:  
  
In [7]: calcular(1,2,3)  
Out[7]: 9  
  
In [8]: calcular([1,2,3],[4,5,6], 2)  
Out[8]: [1, 2, 3, 4, 5, 6, 1, 2, 3, 4, 5, 6]  
  
In [9]: calcular('pera',' y naranja ', 3)  
Out[9]: 'pera y naranja pera y naranja pera y naranja '
```

Tipado con inferencia automática

- Tipos son deducidos al compilar un código
- Haskell permite también la especificación de el tipo asociado a un valor

Acerca del sistema de tipos de Haskell

- Tiene una combinación potente entre estático y con inferencia automática
 - Permite escribir código conciso, seguro y cuyo código tiene gran poder expresivo
- Gracias a este, el compilador es capaz de indicar fallas en el código en una etapa temprana de desarrollo

Tipos básicos

- Char: Caracter **Unicode**
- Bool: Valor binario (lógico)
- Int: Valor entero con signo de tamaño fijo (32 bits generalmente)
- Integer: Valor entero de tamaño no acotado (no se usa tan a menudo como tipo Int)
- Double: Valor con punto flotante (existe el tipo Float, pero se recomienda uso de Double)

Para especificar el tipo de un determinado valor se utilizan ::

```
:type 32323 -- Haskell infiere el tipo
```

```
:type 3 :: Int -- se denomina firma de tipo
```

Tuplas

- Colección de tamaño fijo
- Se delimita mediante (...)
- Puede contener valores de distintos tipos
- ("978-1784394707", "Learning Haskell for Data Analysis", 2015)
- El tipo asociado a una tupla está determinado por número, ubicación y tipo de sus miembros

```
:type (False, 'a')
```

```
:type ('a', False)
```

Listas

- El tipo `list` es polimórfico, debido a que puede contener valores de cualquier otro tipo

```
:type [1,2,3]
```

```
:type [True, True, False]
```

```
:type tail [1, 6, 11]
```

```
:type head [1, 6, 11]
```

```
:type [[True], [False, False, True]]
```

Listas por comprensión

- elementos se describen a través de propiedades que tienen en común
- Ej. Números pares entre 0 y 20

```
[x | x <- [0..20], (x `mod` 2 == 0) ]
```

```
[0,2,4,6,8,10,12,14,16,18,20]
```

- Ej. Números entre 1 y 5 más 3

```
[x+3 | x <- [1,2,3,4,5]]
```

```
[4,5,6,7,8]
```

Clases de tipos

Categorías que agrupan tipos según un comportamiento y soporte de operaciones definidos

Eq: Cualquier tipo que soporte verificación de igualdad ($==$ y $\neq$) entre dos valores (casi todas)

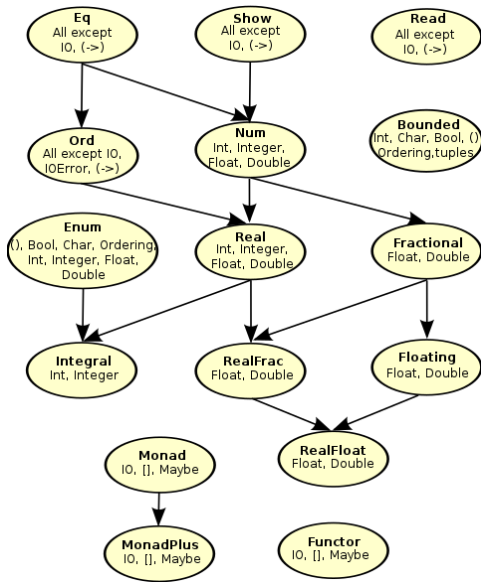
Ord: Cualquier tipo que soporte comparación ($<$, $\leq$, $>$, $\geq$) entre dos valores

Show: Cualquier tipo que pueda ser presentado como String. Es posible verificar que un determinado tipo es parte de esta clase porque sus valores soportan el uso de la función show.

Read: Hace lo opuesto. Es decir, toma un String y retorna un tipo (debe ser miembro de Read). Se puede verificar con la función `read`. Ejemplo: `read "8.2" + 3.8`.

Enum: Tipos cuyos valores están secuencialmente ordenados. Los tipos en esta clase pueden ser usados en rangos de listas, ya que soportan el uso de las funciones `succ` y `pred`. Ejemplo: `succ 'a'`.

Num: Tipos cuyos valores pueden ser usados como números.



Construcción de Funciones en Haskell

Todas las funciones en Haskell siguen 3 reglas. Son estas reglas las que obligan a que una función en Haskell se comporte como una función matemática:

- Deben tomar un argumento
- Deben retornar/generar un valor
- Cada vez que una función es invocada con el mismo argumento, **debe** retornar el mismo valor

Funciones en el interprete vs archivos independientes

- `ghci` no es muy cómodo para escribir funciones, debido a que acepta solamente un conjunto restringido de instrucciones del lenguaje
- Deberemos crear archivos de código, los cuales podrán ser posteriormente **compilados** con `ghc` o cargados directamente desde `ghci`
- Comencemos con una función que sume dos números. Creemos el archivo `calcula_suma.hs` con el siguiente contenido:

-- *función sumar que toma dos números y calcula la suma de estos*

sumar a b = a + b

- Luego, carguemos el archivo dentro de `ghci` y ejecutamos la instrucción:
`ghci> :load calcula_suma.hs`

Algunos aspectos importantes respecto del Ejemplo

- Haskell no tiene una instrucción de retorno
- Una función es una sola expresión y no una **secuencia** de instrucciones
- El **valor de esta expresión es el resultado de la función**
- = representa significado o definición
- En Haskell las variables son solamente un medio para denominar expresiones
 - Una vez asociada a una expresión, su valor no cambia y representa siempre la misma expresión
 - No se asocian con una ubicación en memoria principal
- Construya un archivo denominado `ejemplo_variable_haskell.hs` con el siguiente contenido:

x = 10

x = 11

- Intente cargarlo en `ghci` o compilarlo con `ghc`

Estructura condicional

- Haskell tiene una expresión para If
- Su sintáxis puede ser mostrada mediante el siguiente ejemplo:
 - Construya un archivo denominado `ejemplo.hs` con el contenido:

-- archivo de ejemplo:

```
mayor a b = if a >= b
            then a  -- la tabulación es muy importante
            else b
```

- Cargue el archivo desde `ghci` y ejecute luego: `mayor 23 32`

- La estructura `if` tiene 3 componentes:
 - Expresión de tipo Bool que se denomina ***predicado***
 - Palabra clave `then` seguida de ***otra expresión***`*`. Esta expresión será usada como el valor resultante del `if` cuando el predicado es evaluado como `True`.
 - Palabra clave `else` seguida de ***otra expresión***`*`. Esta expresión será usada como el valor resultante del `if` cuando el predicado es evaluado como `False`.
- Las expresiones que acompañan a `then` y `else` (`*` y `*`) se denominan ramas y **deben** tener el **mismo tipo**
- La clausula `else` es obligatoria

Actividades

- Construya una función que indique si un número dado es par o no lo es.
- Construya una función que tome una lista como parámetro y entregue la cantidad de elementos en ella

Funciones Anónimas

- Se les llama funciones λ
- Son funciones sin un nombre, es decir se usa su propia expresión de definición para invocarlas
- Solamente toman un valor y retornan un valor
- **No** pueden **usarse sin entregarle** un **valor** para su(s) parámetro(s)
- Ejemplo: La función suma puede escribirse como una función lambda: $\lambda x y \rightarrow x + y$
 - Deben entregarse valores a sus parámetros, de otra forma esta expresión genera un error

Polimorfismo en Haskell

- Una función polimórfica es aquella cuya definición permite aplicarla sobre argumentos de distintos tipos
- Por ejemplo: `take (take 3 "lenguaje")`
 - Funciona correctamente sin importar el tipo contenido en la lista
 - Al mirar la firma de esta función, encontramos una variable de tipo `a`

```
Prelude> :type take
```

```
take :: Int -> [a] -> [a]
```

- Esta firma puede leerse como: “`take` toma una lista con elementos del tipo `a` y retorna un valor del mismo tipo `a`”
- Si una función tiene una variable de tipo en su firma, entonces la función es polimórfica

Comentario respecto a las variables de tipo

- Las variables de tipo siempre comienzan con letras minúsculas
- No se pueden confundir con las variables normales debido a que la especificaciones de tipos y de las funciones van separadas

Definiendo tipos de funciones

- Anteriormente hemos visto cómo escribir funciones indicando únicamente variables para cada argumento
- Hemos dejado a Haskell la tarea de inferir los tipos automáticamente
- También es posible especificar esta información al definir la función:

```
-- multiplica dos números
```

```
f1 :: Int -> Int -> Int
```

```
f1 a b = a * b
```

- Pruebe usar la función anterior con la expresión `f1 3 3.0`
- Escriba nuevamente la función como:

```
-- multiplica dos números de cualquier tipo
```

```
-- => es una restricción de tipo
```

```
f1 :: Num a => a -> a -> a
```

```
f1 a b = a * b
```

Definiendo con Patrones

- Se identifican distintos **moldes** para los argumentos o datos de entrada
- La función se define mediante distintos cuerpos (varias definiciones independientes, una por cada **molde** o escenario de uso)
- Por ejemplo, consideremos la función `adivina` que indica si el número entregado como argumento coincide con el número secreto:

```
adivina :: (Integral a) => a -> String
adivina 7 = "¡Encontró el número secreto!"
adivina x = "¡Disculpa, pero no adivinaste!"
```

- Al invocar a la función los patrones se revisan desde arriba hacia abajo, y cuando se produce el calce, se utiliza ese cuerpo.
- Definamos la función factorial de un número de manera recursiva como el producto entre el número y el factorial de su predecesor:

```
factorial :: (Integral a) => a -> a
factorial 0 = 1
factorial n = n * factorial (n - 1)
```

- **Siempre** debe incluirse un ***patrón atrapa-todo*** que aborda cualquier caso no cubierto por los patrones anteriores

Patrones sobre listas

- Un patrón como $x:xs$ asociará la cabeza de la lista a x y el resto de ella a xs

```
head' :: [a] -> a
```

```
head' [] = error "¡No es posible realizar esta operación en una lista vacía!"
```

```
head' (x:_) = x
```

```
head' [7,9,2,10]
```

```
7
```

- Es posible aplicar la misma idea a más de un elemento del tope de una lista

```
primeros3 :: [a] -> [a]
```

```
primeros3 [] = error ";No es posible realizar esta operación en una lista vacía"
```

```
primeros3 (x:y:z:_) = [x,y,z]
```

```
primeros3 [7,9,2,10]
```

```
[7,9,2]
```

- Consideremos ahora el siguiente ejemplo:

```
sumaR :: (Num a) => [a] -> a
```

```
sumaR [] = 0
```

```
sumaR (x:xs) = x + sumaR xs
```

```
sumaR [5,2,1]
```

8

- Otro recurso interesante es @ para poder definir un patrón sin perder la referencia al objeto completo:

```
primeraLetra :: String -> String
```

```
primeraLetra "" = "¡String vacía!"
```

```
primeraLetra palabra@(x:xs) = "La primera letra de " ++ palabra ++ " es " ++
```

```
primeraLetra "Juan"
```

```
"La primera letra de Juan es J"
```

Escoltas en funciones

- Siguen siempre a una función con sus parámetros
- Son expresiones booleans junto con predicados ... similares a las condiciones.
- Admiten multiples condiciones ... bastante parecidos a `if`, `elif` ... `else` en Python
- Se usan ***pipes*** `|` para indicarlos

```
imc :: (RealFloat a) => a -> a -> String -- estado según índice de masa corp
```

```
imc peso estatura
```

```
| peso / estatura ^ 2 <= 18.5 = "¡Muy bajo!"
```

```
| peso / estatura ^ 2 <= 25.0 = "¡Se supone que es normal!"
```

```
| peso / estatura ^ 2 <= 30.0 = "¡Mejor no digo que estás un poco pasado"
```

```
| otherwise = "¡Felicitaciones! sacaste el premio gordo."
```

```
putStrLn (imc 84 1.77)
```

¡Mejor no digo que estás un poco pasado!

¿Como implementaría la función max entre dos números usando escoltas?

Usando Where dentro de las funciones

- En ocasiones se repiten expresiones múltiples veces
- Por ejemplo, en el caso de `imc` con la expresión `peso / estatura ^ 2`
- La redundancia en este caso no es algo deseable.. dificulta interpretabilidad y mantenimiento del código
- **Existe** una manera de asociar estas expresiones a un solo nombre mediante la instrucción `where`

```
imc :: (RealFloat a) => a -> a -> String -- estado según índice de masa corp
imc peso estatura
  | valor <= flaco = ";Muy bajo!"
  | valor <= medio = ";Se supone que es normal!"
  | valor <= pasado = ";Mejor no digo que estás un poco pasado!"
  | otherwise = ";Felicitaciones! sacaste el premio gordo."
where valor = peso / estatura ^ 2
      flaco = 18.5
      medio = 25.0
      pasado = 30.0
```

```
putStrLn (imc 84 1.77)
```

```
`¡Mejor no digo que estás un poco pasado!`
```

Si decidimos cambiar la manera de calcular el índice, solo modificamos el código en un solo lugar.

Funciones de Alto Nivel

- Nos referimos a funciones cuyos argumentos son funciones y su retorno también
- Es importante encerrar entre () el primer parámetro, debido a que `->` es asociativo por derecha

```
aplicar2veces :: (a -> a) -> a -> a
```

```
aplicar2veces f x = f (f x)
```

```
aplicar2veces (\p -> 2 * p) 5
```

```
aplicar2veces (+ 3) 10
```

```
aplicar2veces (++ " waka") "waka"
```

```
20
```

```
16
```

```
"waka waka waka"
```

Algunas funciones nativas

- `zip`: Combina los elementos de dos listas en una lista de tuplas
- `zipWith`: Combinar dos listas usando una función binaria
- `map`: Toma una función y una lista para luego aplicar la función a cada elemento
- `filter`: Toma un predicado (función que retorna True o False) y una lista, entregando finalmente solo aquellos elementos que satisfacen el predicado

```
zip [1,2,3] [4,5,6]
zipWith (+) [1,2,3] [10,20,30]
zipWith replicate [1,2,3] [10,20,30]
map (replicate 2) [1..5]
filter even [1..10]

[(1,4),(2,5),(3,6)]

[11,22,33]

[[10],[20,20],[30,30,30]]

[[1,1],[2,2],[3,3],[4,4],[5,5]]

[2,4,6,8,10]
```

Construcción de un programa ejecutable

```
-- ejemplo.hs Este es mi primer código ejecutable de ejemplo  
main = do  
    print "¡Hola, estoy corriendo!"
```

Línea con comentario
descriptivo acerca del contenido del archivo

Comienzo de
función principal

Contenido de la función principal

Introducción

- Cada LP tiene sus tipos de dato básicos
- Frecuentemente es posible resolver un gran número de problemas con estos tipos (Int, Float o String por ejemplo)
- A veces, se adaptan los tipos nativos de manera un tanto **forzada**
- Ejemplo: Se desea representar un círculo mediante su radio y ubicación en el plano 2D.
 - Una alternativa es usar tuplas como (0.501, 1.208, 4)
 - No resulta muy intuitiva y puede generar confusión respecto a lo que hay en cada componente
 - Se puede hacer bastante incómodo acceder a cada atributo usando calce de patrones
- Se pueden hacer sinónimos de tipos ya existente

```
type NombrePila = String
```

```
type Apellido = String
```

```
type Edad = Int
```

```
-- función que entrega un String con el nombre, apellido y edad de una persona
```

```
strInfoPersona :: NombrePila -> Apellido -> Apellido -> Edad -> String
```

```
strInfoPersona n a1 a2 e = a1 ++ " " ++ a2 ++ ", " ++ n ++ "/" ++ (show e)
```

```
strInfoPersona "juan" "zamora" "osorio" 37
```

```
"zamora osorio, juan/ 37 annos"
```

- No solo se pueden renombrar tipos de a uno

```
type TipoPersona = (NombrePila, Apellido, Apellido, Edad) -- importante la m
```

```
obtenerPrimerApellido :: TipoPersona -> Apellido
obtenerPrimerApellido (_, a, _, _) = a

obtenerPrimerApellido ("juan", "zamora", "osorio", 37)

"zamora"
```

- Se pueden definir otros **nuevos**
 - En otro lenguaje se hubieran usado los tipos nativos para definir uno nuevo

data Computador = Escritorio | Portatil -- *el tipo Computador tendrá 2 const*

- En este caso:
 - Computador corresponde al **constructor de tipo**
 - Este constructor también puede tomar argumentos
 - Escritorio y Portatil son constructores de datos. Son usados para crear instancias concretas del tipo
- Al separar los constructores de dato con | se indica que el tipo Computador puede ser una instancia de Escritorio o Portatil

Ejercicio

- Defina el tipo Persona e incluya el tipo de sangre en su información personal
 - Se compone de 2 elementos: tipo ABO (A, B, AB, O) y valor Rhesus (Rh + o Rh -)
- Escriba una función que tome dos instancias del tipo sanguíneo y retorne True cuando es posible la transfusión entre ambos.
 - A puede donar a A y AB
 - B puede donar a B y AB
 - AB puede donar solo a AB
 - O puede donar a cualquiera ""

```
type TipoABO = String
type Rh = String
```

```
-- En este caso no podemos colocar data TipoSangre = TipoABO | Rh , ya que no se puede
data TipoSangre = TipoSangre TipoABO Rh -- este constructor de datos con argumentos
```

Primero definimos una función que nos permita obtener el tipo ABO de un dato TipoSangre

```
obtenerAB0 :: TipoSangre -> TipoAB0  
obtenerAB0 (TipoSangre x _) = x
```

ahora podemos crear un dato de este tipo con el constructor de datos y también usar la función

```
pac1 :: TipoSangre  
pac1 = TipoSangre "A" "POS"
```

```
pac2 :: TipoSangre  
pac2 = TipoSangre "O" "NEG"
```

```
pac3 :: TipoSangre  
pac3 = TipoSangre "AB" "POS"
```

obtenerAB0 (TipoSangre "AB" "POS") -- uso del constructor de datos

obtenerAB0 pac1

obtenerAB0 pac2

obtenerAB0 pac3

"AB"

"A"

"O"

"AB"

Ahora definamos la función siguiendo las condiciones de transfusión planteadas en el enunciado

```
puedeDonar :: TipoSangre -> TipoSangre -> Bool
puedeDonar paci pacj
  | aboi == "A" && (aboj == "A" || aboj == "AB") = True
  | aboi == "B" && (aboj == "B" || aboj == "AB") = True
  | aboi == "AB" && aboj == "AB" = True
  | aboi == "O" = True
  | otherwise = False
  where aboi = obtenerABO paci
        aboj = obtenerABO pacj
```

```
puedeDonar pac1 pac2  
puedeDonar pac2 pac1  
puedeDonar pac2 pac3  
puedeDonar pac1 pac3  
puedeDonar pac3 pac1
```

False

True

True

True

False

Solución alternativa sin usar Strings para TipoABO Y RH

```
data TipoABO = A | B | AB | O
```

```
data TipoRh = Pos | Neg
```

-- En este caso no podemos colocar data TipoSangre = TipoABO | Rh , ya que r

```
data TipoSangre = TipoSangre TipoABO TipoRh -- este constructor de datos con
```

Para poder mostrar datos con estos tipos necesitamos una función por tipo:

```
mostrarABO :: TipoABO -> String
```

```
mostrarABO A = "A"
```

```
mostrarABO B = "B"
```

```
mostrarABO AB = "AB"
```

```
mostrarABO O = "O"
```

```
mostrarRh :: TipoRh -> String
```

```
mostrarRh Pos = "+"
```

```
mostrarRh Neg = "-"
```

```
mostrarTipoSangre :: TipoSangre -> String
```

```
mostrarTipoSangre (TipoSangre abo rh) = mostrarABO abo ++ mostrarRh rh
```

<>

Agosto, 2024. 70

nuestra función puedeDonar ahora será definida usando patrones

```
puedeDonar :: TipoSangre -> TipoSangre -> Bool
puedeDonar (TipoSangre 0 _) _ = True
puedeDonar _ (TipoSangre AB _) = True
puedeDonar (TipoSangre A _) (TipoSangre A _) = True
puedeDonar (TipoSangre B _) (TipoSangre B _) = True
puedeDonar _ _ = False
```

```
puedeDonar pac1 pac2  
puedeDonar pac2 pac1  
puedeDonar pac2 pac3  
puedeDonar pac1 pac3  
puedeDonar pac3 pac1
```

False

True

True

True

False

Alternativa ultra-cómoda para definir tipos

- Sintáxis de registro usando {}

```
data Estudiante = Estudiante {  
    primerNombre :: String,  
    apellidoPat  :: String,  
    apellidoMat  :: String,  
    edad        :: Int,  
    estatura    :: Float,  
    carrera     :: String  
} deriving (Show)
```

Usemos el modo interactivo para crear un objeto de este tipo

```
let e1 = Estudiante "Juan" "Zamora" "Osorio" 19 1.75 "Ingenieria Estadistica"
```

- Haskell automáticamente genera métodos de acceso a los campos

```
edad e1
```

```
estatura e1
```

```
19
```

```
1.75
```

Porqué definir tipos propios?

- Mejora la legibilidad del código
- Permite que el compilador verifique que se usa el tipo adecuado y no otro que accidentalmente calce, pero que genere un error posterior.

Introducción

- Transversal a cualquier lenguajes son los requerimientos de entrada y salida
- **Entrada:** Solicitar ingreso externo de datos (desde teclado o archivos en el disco)
- **Salida:** Desplegar datos en algún medio (pantalla o archivos en el disco)
- Al depender de el ingreso de datos se viola la Transparencia referencial
 - Haskell resuelve esto con el tipo IO
 - Cualquier función que use IO será marcada como proveniente de IO

Revisemos el ejemplo:

```
funcionQueSuma1 val1 val2 = (val1 + val2 + 1)
```

```
funcionQueSumaX val1 val2 = do  -- do permite secuenciar instrucciones de IO
    putStrLn "Ingresar un número:"
    xIn <- getLine
    let x = read xIn
    return ((val1 + val2 + x))
```

- Será posible ejecutar `ghci> (funcionQueSuma1 4 5) + (funcionQueSuma1 2 5)`
- Pero no se podrá `ghci> (funcionQueSumaX 4 5) + (funcionQueSumaX 4 5)`

Las funciones putStrLn y getLine

- Ambas retornan lo que se denomina como acciones de entrada y salida (**IO actions**)
 - Esto quiere decir que son **funciones** que violan alguno de los 3 principios: todas toman un valor, todas retornan un valor y mismo argumento => mismo resultado.

:t putStrLn

:t getLine

- Esto quiere decir que no entregan valor alguno (putStrLn) y que no reciben argumento (getLine)
- El valor generado en cualquier función marcada como IO no puede ser usado fuera de ella

El bloque do

- Permite especificar secuencias de instrucciones con entrada y salida.
- por ejemplo

```
do {putStrLn "Linea 1"; putStrLn "Linea 2"; putStrLn "Linea 3"}
```

Linea 1

Linea 2

Linea 3

- Con ellos podemos **escapar** de la prisión de las funciones marcadas como IO
- Cuando el resultado de una función marcada como IO se usa en una asignación, empleamos <-
- Para crear variables que reciben valores de funciones no marcadas como IO se usa =

Revisemos este ejemplo

```
digahola :: String -> String
```

```
digahola n = "Hola" ++ " " ++ n ++ "!"
```

```
main :: IO ()
```

```
main = do
```

```
    putStrLn "Indiqueme su nombre por favor:"
```

```
    nom <- getLine -- recordar que no recibe argumentos
```

```
    let mensaje = digahola nom -- dentro del bloque 'do' se usa como si fuer
```

```
    putStrLn mensaje
```

- >> runghc archivo_io.hs